🚀 DAY 9 — MULTI-STEP AGENT (Excel + RAG together)

Now we upgrade your system to:

User Query

↓

Planner (LLM decides steps ✅)

↓

Step 1 → Excel

Step 2 → RAG

Step 3 → Combine

↓

Final Answer ✅ 🔥

✅ 🎯 WHAT THIS ENABLES

You can now answer:

"Find cars with highest price and explain trends from document"

👉 System flow:

1. Excel → find highest price ✅

2. RAG → get explanation from PDF ✅

3. Combine → final answer ✅

✅ ✅ STEP 1 — Add Multi-step Planner

✅ Create: planner.py in app folder

```
from models.llm import generate

class Planner:

    def create_plan(self, query):

        prompt = f"""

You are a planning agent.
```

```
Break the query into steps.

Available tools:

- EXCEL → for dataset queries

- RAG → for document knowledge

Return format:

Step 1: <tool>: <task>

Step 2: <tool>: <task>

...

Examples:

Q: average price

Step 1: EXCEL: calculate average price

Q: max price and explain trend

Step 1: EXCEL: find maximum price

Step 2: RAG: explain pricing trends

Now: Q: {query}

"""         response = generate(prompt)
        return self.parse_plan(response)
    def parse_plan(self, text):
        steps = []
        for line in text.split("\n"):
            if "Step" in line:
                parts = line.split(":")
                if len(parts) >= 3:
                    tool = parts[1].strip()
                    task = parts[2].strip()
                    steps.append((tool, task))
        return steps
```

✅ ✅ STEP 2 — Update Agent

✅ Replace agent.py

```
from models.llm import generate
from app.planner import Planner


class Agent:
    def __init__(self, rag_system, excel_tool=None):
        self.rag = rag_system
        self.excel = excel_tool
        self.planner = Planner()

    # ✅ Multi-step execution
    def run(self, query):
        # Step 1: get plan from planner
        plan = self.planner.create_plan(query)
        # fallback if planner fails
        if not plan:
            plan = [("EXCEL", query)]
        print("FINAL PLAN:", plan)
        results = []

        # Step 2: execute steps
        for i, (tool, task) in enumerate(plan, 1):
            print(f"Executing Step {i}: {tool} → {task}")
            if tool == "EXCEL" and self.excel:
                result = self.excel.smart_query(task, generate)
```

```
            elif tool == "RAG":
                docs = self.rag.search(task)
                result = "\n".join(docs[:3])
            else:
                result = "Unknown tool"
            print(f"STEP {i} RESULT:", result)
            results.append(f"Step {i} Result:\n{result}")

        # Step 3: final answer (keep Claude style)
        final_prompt = f"""
Use these step results to answer the question:
{results}
Question: {query}
"""
        final_answer = generate(final_prompt)
        return final_answer
```

✅ ✅ STEP 3 — TEST

🔥 Try queries:

>> maximum price and explain from document

>> average price and give insights

>> cars with lowest km and explain usage trend

✅ ✅ OUTPUT FLOW

Example:

PLAN:

[('EXCEL', 'find maximum price'),

('RAG', 'explain pricing trends')]

Answer:

Max price = 160.0.

According to the document...

✅ ✅ WHAT YOU JUST BUILT

🔥 This is BIG:

✅ Single-step agent → BEFORE

✅ Multi-step reasoning agent → NOW ✅ 🔥

🧠 INDUSTRY TERM

You now have:

✅ Planner + Tool Executor Agent

Used in:

- AutoGPT
- LangGraph
- Microsoft Copilot
- Advanced LangChain

🚀 NEXT (Day 10 preview)

We can upgrade to:

✅ Memory (chat history)

✅ Context-aware reasoning

✅ Autonomous agents

✅ SUMMARY

You now have:

- ✔ RAG ✅
- ✔ Excel Agent ✅
- ✔ Routing ✅
- ✔ Multi-step Planner ✅ 🔥

💬 Say Day 10 when ready 🚀